

JSON

Step-by-Step Strategy

1. Understand the JSON Structure

You typically have:

- A root object with league-level data (e.g., `name`, `season`, `start_date`, etc.).
- An array of nested objects (e.g., `matches`) within each root object.

2. Understand the Table Schema

From your image:

- `FootballLeague` has top-level fields.
- `Matches` references `FootballLeague` with a foreign key (`league_id`) and stores match details.

3. Load JSON into SQL

Use DuckDB's `read_json()` function:

```
SELECT * FROM read_json('football.json', columns = {...});
```

Make sure to define the **format** (`'array'`, `'unstructured'`, etc.) if needed.

Tips for Writing SQL DML

Insert into `FootballLeague`

You're inserting **top-level data**:

```
INSERT INTO FootballLeague (league, season, start_date, end_date, country)
SELECT name, season, start_date, end_date, country
FROM read_json('football.json', columns={
  name: 'VARCHAR',
```

```
season: 'VARCHAR',
start_date: 'DATE',
end_date: 'DATE',
country: 'VARCHAR'
});
```

Insert into **Matches**

You're dealing with **arrays inside the JSON**. You need to:

- Use `UNNEST()` on JSON arrays.
- Extract fields like `round`, `team1`, etc. using `>>` or path syntax.
- Join each match back to its corresponding league using `league` and `season`.

```
INSERT INTO Matches (
  round, match_date, match_time, home_team, away_team, score, league_id
)
SELECT
  unnest(matches->>'$[*].round'),
  unnest(matches->>'$[*].date'),
  unnest(matches->>'$[*].time'),
  unnest(matches->>'$[*].team1'),
  unnest(matches->>'$[*].team2'),
  unnest(matches->>'$[*].score'),
  (
    SELECT id FROM FootballLeague f
    WHERE f.league = name AND f.season = season
  )
FROM read_json('football.json', columns={
  name: 'VARCHAR',
  season: 'VARCHAR',
  matches: 'JSON'
});
```

Conceptual Tips

1. **Always insert parent tables first** (`FootballLeague` before `Matches`).
2. Use `UNNEST()` or path expressions to flatten JSON arrays.
3. Use subqueries to resolve **foreign keys** (link `league_id`).
4. You can preview JSON using `SELECT * FROM read_json(...)` to understand the structure before writing complex queries.
5. **Normalize**: Avoid duplication by linking nested data back to the parent using unique fields (e.g., name + season).